ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Nguyễn Vũ Đức Anh

BÀI TẬP 10 – LẬP TRÌNH THUẬT TOÁN BANKER VÀ THUẬT TOÁN PHÁT HIỆN BẾ TẮC

Ngành: Mạng máy tính và truyền thông dữ liệu

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

HÀ NỘI - 2026

Nguyễn Vũ Đức Anh

BÀI TẬP 10 – LẬP TRÌNH THUẬT TOÁN BANKER VÀ THUẬT TOÁN PHÁT HIỆN BẾ TẮC

Ngành: Mạng máy tính và truyền thông dữ liệu

Supervisor:
Co-Supervisor:

HÀ NỘI - 2026

TÓM TẮT

Tóm tắt: Lập trình thuật toán Banker và thuật toán phát hiện bế tắc bằng Java. Nhận input là trạng thái các tài nguyên, mức sử dụng tài nguyên của tiến trình và các yêu cầu tài nguyên. Trả về output là thông báo về trạng thái của hệ thống (an toàn/ không an toàn) (bế tắc/ không bế tắc) và in ra chuỗi an toàn.

I. Class SystemOS:

Class SystemOS để tạo đối tượng hệ thống bao gồm thuật toán được lựa chọn chạy, biến numProcess lưu số tiến trình trong hệ thống, numResource lưu số loại tài nguyên. Các mảng 2 chiều allocation thể hiện số tài nguyên được cấp cho tiến trình, mảng demand là mảng lưu số lượng tài nguyên tối đa của tiến trình đối với thuật toán Banker và lưu request đối với thuật toán phát hiện. Mảng available lưu tài nguyên sẵn có, mảng request lưu yêu cầu tài nguyên từ tiến trình trong thuật toán Banker. Mảng need tính số tài nguyên mà tiến trình có thể cần.
Chuỗi hằng MODE_BANKER và MODE_DETECTION dùng để thể hiện thuật toán Banker và thuật toán phát hiện.

```
public class SystemOS {  2 usages  VuDucAnhNguyen *
    private String mode;  5 usages
    private int numProcess;  16 usages
    private int numResource;  25 usages

    private int[][] allocation;  8 usages
    private int[][] demand;  4 usages
    private int[] available;  7 usages
    private int[] request;  15 usages
    private int[][] need;  6 usages

    public static final String MODE_BANKER = "BANKER";  3 usages
    public static final String MODE_DETECTION = "DETECTION";  1 usage
```

Hàm calculateNeed() thực hiện tính mảng need theo công thức need = demand – allocation.

```
private void calculateNeed() {  1 usage  VuDucAnhNguyen
    for (int i = 0; i < numProcess; i++) {
        for (int j = 0; j < numResource; j++) {
            need[i][j] = demand[i][j] - allocation[i][j];
        }
    }
}
```

Hàm readFile() đọc file input, từ đầu tiên là thuật toán được lựa chọn. 2 số tiếp theo lần lượt là số tiến trình và số loại tài nguyên. Thực hiện cấp phát mảng, numResource số tiếp theo là mảng available. numProcess dòng có numResource số tiếp theo là mảng demand. numProcess dòng có numResource số tiếp theo là mảng allocation. Đối với thuật toán Banker, numResource+1 số tiếp theo là tài nguyên mà tiến trình request[numResource] yêu cầu và thực hiện tính need. Kết thúc chương trình và thông báo lỗi nếu không tìm thấy file hoặc sai format.

```java
private void readFile (String filePath) {  1 usage  VuDucAnhNguyen
    try {
        File inputFile = new File (filePath);
        Scanner reader = new Scanner(inputFile);

        if (reader.hasNextLine()){
            mode = reader.next();
        }

        if (reader.hasNextLine()){
            this.numProcess = reader.nextInt();
            this.numResource = reader.nextInt();
        }

        this.allocation = new int[numProcess][numResource];
        this.demand = new int[numProcess][numResource];
        this.available = new int[numResource];
        this.request = new int[numResource + 1];

        for (int j = 0; j < numResource; j++){
            available[j] = reader.nextInt();
        }

        for (int i = 0; i < numProcess; i++){
            for (int j = 0; j < numResource; j++){
                demand[i][j] = reader.nextInt();
            }
        }
```

```java
        for (int i = 0; i < numProcess; i++) {
            for (int j = 0; j < numResource; j++) {
                allocation[i][j] = reader.nextInt();
            }
        }

        if (mode.equals(MODE_BANKER)) {
            for (int j = 0; j <= numResource; j++){
                request[j] = reader.nextInt();
            }
        }

        if (mode.equals(MODE_BANKER)) {
            this.need = new int[numProcess][numResource];
            calculateNeed();
        }

        reader.close();
    } catch (FileNotFoundException e) {
        System.out.println("File not found!");
        System.exit( status: 1);
    } catch (Exception e){
        System.out.println("Invalid input format!");
        System.exit( status: 1);
    }
}
```

Hàm safeAlgorithm() là thuật toán an toàn trong Banker, trả về true nếu hệ thống an toàn và false nếu không an toàn. thực hiện tạo bản sao work từ available và tạo mảng false với tất cả phần tử là false thể hiện tiến trình đã hoàn thành. Thưc hiện tìm tiến trình có finish = false và đủ tài nguyên được yêu cầu thì cấp phát, thu hồi tài nguyên, finish = true và lưu vào mảng hoàn thành. Nếu không còn tiến trình nào thỏa mãn điều kiện trên mà vẫn còn tiến trình false thì lúc này hệ thống bế tắc (hệ thống sau cấp phát không an toàn). Nếu không còn tiến trình false thì hệ thống an toàn và in ra thứ tự hoàn thành tiến trình.

```java
private boolean safeAlgorithm() {  1 usage  VuDucAnhNguyen
    ArrayList<Integer> safe = new ArrayList<>();
    int[] work = available.clone();

    boolean[] finish = new boolean[numProcess];
    for (int i = 0; i < numProcess; i++) {
        finish[i] = false;
    }

    while (true) {
        boolean found = false;

        for (int i = 0 ; i < numProcess; i++) {
            if (!finish[i]) {
                boolean runnable = true;
                for (int j = 0; j < numResource; j++) {
                    if (need[i][j] > work[j]){
                        runnable = false;
                        break;
                    }
                }

                if (runnable) {
                    for (int j = 0; j < numResource; j++) {
                        work[j] += allocation[i][j];
                    }

                    finish[i] = true;
                    safe.add(i);
                    found = true;
```

```java
                }
            }
        }

        if (!found) {
            break;
        }
    }

    if (safe.size() != numProcess) {
        System.out.println("System is unsafe");
        return false;
    } else {
        System.out.println("System is safe: " + safe);
        return true;
    }
}
```

Hàm requestAlgorithm() là thuật toán yêu cầu trong Banker, trả về true nếu có thể nhận yêu cầu và false khi yêu cầu không hợp lệ. Kiểm tra yêu cầu có lớn hơn số tài nguyên tối đa được cấp cho tiến trình hoặc lớn hơn tài nguyên sẵn có hay không và giả định cấp phát.

```java
private boolean requestAlgorithm() {  1 usage  VuDucAnhNguyen
    for (int j = 0; j < numResource; j++){
        if (request[j] > need[request[numResource]][j]) {
            System.out.println("Request more than max!");
            return false;
        }
    }

    for (int j = 0; j < numResource; j++){
        if (request[j] > available[j]) {
            System.out.println("Resources not ready!");
            return false;
        }
    }

    for (int j = 0; j < numResource; j++){
        available[j] -= request[j];
        allocation[request[numResource]][j] += request[j];
        need[request[numResource]][j] -= request[j];
    }
    return true;
}
```

Hàm Banker() là thuật toán Banker. Kiểm tra yêu cầu hợp lệ, nếu có thực hiện chạy thuật toán an toàn. Thuật toán an toàn trả về true thì yêu cầu được cho phép, nếu không yêu cầu bị từ chối và thu hồi tài nguyên được cấp phát giả định.

```java
private void Banker(){  1 usage  VuDucAnhNguyen
    System.out.println("Banker algorithm:");

    if (requestAlgorithm()){
        if (safeAlgorithm()) {
            System.out.println("Request approved!");
        } else {
            System.out.println("Request denied!");
            for (int j = 0; j < numResource; j++){
                available[j] += request[j];
                allocation[request[numResource]][j] -= request[j];
                need[request[numResource]][j] += request[j];
            }
        }
    }
}
```

Hàm Detection() là thuật toán phát hiện bế tắc.Thực hiện tạo bản sao work từ available và tạo mảng false với tất cả phần tử là false thể hiện tiến trình đã hoàn thành. Thực hiện tìm tiến trình có finish = false và đủ tài nguyên được yêu cầu thì cấp phát, thu hồi tài nguyên, finish = true và lưu vào mảng hoàn thành. Nếu không còn tiến trình nào thỏa mãn điều kiện trên mà vẫn còn tiến

trình false thì lúc này hệ thống bế tắc và in ra các tiến trình gây ra bế tắc. Nếu không còn tiến trình false thì hệ thống an toàn và in ra thứ tự hoàn thành tiến trình.

```java
private void Detection() {  1 usage  VuDucAnhNguyen
    System.out.println("Deadlock detection algorithm:");

    ArrayList<Integer> result = new ArrayList<>();
    int[] work = available.clone();

    boolean[] finish = new boolean[numProcess];
    for (int i = 0; i < numProcess; i++) {
        finish[i] = false;
    }

    while (true) {
        boolean found = false;

        for (int i = 0; i < numProcess; i++) {
            if (!finish[i]) {
                boolean runnable = true;
                for (int j = 0; j < numResource; j++) {
                    if (demand[i][j] > work[j]) {
                        runnable = false;
                        break;
                    }
                }

                if (runnable) {
                    for (int j = 0; j < numResource; j++) {
                        work[j] += allocation[i][j];
                    }

                    finish[i] = true;
                    result.add(i);
                    found = true;
                }
            }
        }

        if (!found) {
            break;
        }
    }

    if (result.size() == numProcess) {
        System.out.println("System is safe: " + result);
    } else {
        System.out.println("System is deadlock at:");
        for (int i = 0; i < numProcess; i++) {
            if (!finish[i]){
                for (int j = 0; j < numResource; j++) {
                    if (allocation[i][j] != 0) {
                        System.out.println("Process " + i);
                        break;
                    }
                }
            }
        }
    }
}
```

Hàm Run() nhận input là đường dẫn tới file input. Thực hiện đọc file, và chạy thuật toán file yêu cầu, báo lỗi nếu thuật toán không tồn tại.

```
    public void Run (String filePath) {  1 usage  VuDucAnhNguyen *
        readFile(filePath);

        if (mode.equals(MODE_BANKER)) {
            Banker();
        } else if (mode.equals(MODE_DETECTION)) {
            Detection();
        } else {
            System.out.println("Invalid algorithm");
        }
    }
}
```

II. Class Main:

Nơi bắt đầu chương trình, khởi tạo SystemOS và truyền vào đường dẫn tới file input.

```
public class Main {  VuDucAnhNguyen
    public static void main (String[] args) {  VuDucAnhNguyen
        SystemOS system2 = new SystemOS();
        system2.Run( filePath: "src/input.txt");
    }
}
```

III. Chạy và kết quả:

Testcase 1:

Main.java input.txt SystemOS.java

```
DETECTION 5 3
0 0 0
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
```

```
Deadlock detection algorithm:
System is safe: [0, 2, 3, 4, 1]

Process finished with exit code 0
```

Testcase 2:

Main.java × input.txt × SystemOS.java

```
BANKER 5 3
3 3 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
1 0 2 1
```

```
Banker algorithm:
System is safe: [1, 3, 4, 0, 2]
Request approved!

Process finished with exit code 0
```

Testcase 3:

Main.java input.txt SystemOS.java

```
BANKER 5 3
3 3 2
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
3 3 0 4
```

```
Banker algorithm:
System is unsafe
Request denied!

Process finished with exit code 0
```

Testcase 4:

Main.java | input.txt | SystemOS.java

```
DETECTION 5 3
0 0 0
0 0 0
2 0 2
0 0 1
1 0 0
0 0 2
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
```

```
Deadlock detection algorithm:
System is deadlock at:
Process 1
Process 2
Process 3
Process 4

Process finished with exit code 0
```